Week 5: Building with AI

AI Development Course

Tested with Claude Code 2.1.296, superpowers 6.4.1 and Python 3.12.3 on Ubuntu Linux (in a container), 12 October 2026, without signing in; and signed in on Windows (WSL) with Claude Code 2.1.312 and superpowers 7.0.0, 29 October 2026. GitHub and the Mac are being tested.

Today

- What building with AI means
- Install Superpowers again
- Pick your tools, and read every install prompt
- Plan the first three steps with `writing-plans`
- Build with `subagent-driven-development`
- Usage limits
- Finish: check it yourself, pull request, merge, pull
- The assignment, and next week

What building with AI means

Claude writes the code. **You** decide what gets built, read what it asks to do, and check the result.

- **Small steps:** one build step at a time
- **Tests first:** a test checks the code's answer against an answer worked out by hand. Claude writes it, watches it **fail**, then writes code until it **passes**
- **A review after every step:** a second Claude, which didn't write the code, checks it against the plan

Superpowers' rule: "NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST".

The Superpowers skills you use

Skill	What it does
`writing-plans`	Your three build steps become a plan file: tasks, tests, files
`subagent-driven-development`	A fresh Claude builds each task; another reviews it
`test-driven-development`	The tests-first rule each task follows
`using-git-worktrees`	Offers a separate folder: this week, say **No**
`finishing-a-development-branch`	Runs the tests, then push and pull request

More depth: advanced/grill-me-superpowers/. Optional extra: advanced/beads/.

Install Superpowers, then branch

In your terminal: `cd ~/semester-project && claude`, then `manual mode on`. In Claude Code:

```
/plugin install superpowers@claude-plugins-official
```

Install for you (user scope), then `/reload-plugins` if asked. Check: `claude plugin list` (second terminal window) shows `Version: 7.0.0` or higher.

Branch now, before Claude writes anything. `Show me git status.` must say `nothing to commit, working tree clean`. Then:

```
Switch to main and pull the latest changes from GitHub. Then make a
new branch called build-first-steps and switch to it.
```

Branch name taken? Use build-first-steps-2.

Pick your tools

```
Read PLAN.md. Recommend the simplest programming language and tools
for building its first three build steps. I'm not a programmer. It
must run on a Mac and on Windows in WSL (Ubuntu), and my laptop is
[...]. Prefer what's already installed, as few installs as possible,
and tests I can run with one command. Check what's already installed
first. Before you run any install command, explain in one or two
sentences what it installs and why, and wait for me to say go. Don't
create or change any files yet.
```

Usually **Python** or **Node.js**. Something else? "Can I do this with Python or Node.js instead?"

Read every install prompt

Check three things: did Claude **explain it first**? Is it **for your laptop**? Is it on the **No** list?

Laptop	Fine	Say No to
Mac, Apple chip	`brew install`, official `.pkg`	`apt`, `winget`, `.exe`
Intel Mac	official `.pkg`	`brew`, `apt`, `winget`, `.exe`
Windows (WSL)	`sudo apt install`, nvm	`brew`, `winget`, PowerShell, `.exe`, `.pkg`

Always **No**: `--force`, `rm -rf`, `--break-system-packages`, `sudo pip`, any `curl | bash` (one exception: nodejs.org's nvm line on WSL, typed by you).

WSL `sudo`: choose No and **run it yourself** in a second Ubuntu window (not with `!` in Claude Code). Mac `.pkg`: asks in its own window. Never type a password into Claude.

Official install links

Check first, in a second terminal window: `python3 --version`, `node --version`

	Python	Node.js (LTS)
Mac, Apple chip or Intel	3.9 or higher already? Use it. Else python.org/downloads/macos, **macOS installer** (3.15.0 when we checked)	nodejs.org/en/download, **macOS Installer** (being tested)
Windows (WSL)	Missing? `sudo apt update && sudo apt install -y python3 python3-venv` (3.12.3 in our test)	nvm, from nodejs.org: type it yourself, one line at a time

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash
\. "$HOME/.nvm/nvm.sh"
nvm install 24
```

After an install: `/exit`, quit and reopen your terminal, `cd ~/semester-project && claude`, `manual mode on`, then "We chose Python with built-in unittest, no installs. It's installed. We're on the build-first-steps branch. Don't write any code yet: we plan first." Check `--version` in a second window.

Plan the first three steps

Keep tool folders out of git first:

```
Add .venv/, node_modules/, __pycache__/ and .claude/worktrees/ to
.gitignore (create it if it doesn't exist), and commit .gitignore on
this branch.
```

Then:

```
/superpowers:writing-plans Write an implementation plan for only the
first three build steps in PLAN.md (the section "First three build
steps"), one task per step, nothing else. Use the language and tools
we just chose. Build each task test first, with tests I can run with
one command, and include at least one example worked out by hand.
I'm not a programmer, so keep each task small.
```

Saved to `docs/superpowers/plans/YYYY-MM-DD-<name>.md`

Reading the plan

- **Goal**, **Architecture**, **Tech Stack**, **Spec** (your `PLAN.md`)
- **Global Constraints**, **Review Focus** (the risky cases)
- `### Task 1:` ... each with **Files** and checkbox **steps**:
  write the failing test, see it fail, write the code, see it pass, commit

Check:

- **exactly three tasks**, one per build step
- every task writes its **test first**
- an example **worked out by hand**, with its answer
- nothing from "Not in this version"

Then work out **your own** example by hand. You need it in step 7.

The deal screener's plan: Task 2, the buy-box check

`example-project/docs/superpowers/plans/2026-10-12-first-three-steps.md`

Hand-worked first: listing A cash-on-cash **5.744%** (fails the 10% rule); listing B cash flow **$430.53** (passes).

Step	What happens
1. Write the failing test	`tests/test_buybox.py`, e.g. `test_listing_a_fails_on_cash_on_cash_only`
2. Run it, see it fail	`python3 -m unittest`
3. Write the code	`dealscreener/buybox.py`
4. Run it, see it pass	`python3 -m unittest`
5. Commit	`git commit -m "Add the buy-box check with a reason for each rule"`

```
Step 2: ModuleNotFoundError: No module named 'dealscreener.buybox'
Step 4: Ran 21 tests ... OK
```

The handoff question

"Plan complete and saved to ... Which execution approach would you prefer?"

Choice	Means
Subagent-driven	Fresh Claude per task, a review after each task, then a whole-branch review. Most thorough, uses more
Native ("inline")	Claude builds it all itself; one review at the end. Cheaper

Choose Subagent-driven, even if it recommends Native. First type `/usage` and write down what it shows (step 6a): your 'before' number.

```
Subagent-driven. Before you start building, commit the plan file on
this branch if it isn't committed yet.
```

The build

Worktree question? "Would you like me to set up an isolated worktree?"

```
No. Work here, on the build-first-steps branch.
```

Then, for each task, without stopping in between:

1. **Implementer:** test, see it fail, code, see it pass, commit. Reports `DONE`, `DONE_WITH_CONCERNS`, `NEEDS_CONTEXT` or `BLOCKED`
2. **Reviewer:** checks against the plan; finds Critical, Important, Minor
3. **Fixes:** Critical or Important only; up to 5 rounds

At the end: a whole-branch review, and "Rulings I made". **Note one review finding.**

Yes or No during the build

Claude asks to	Choose
Create or edit files in your project	**Yes**
Run the tests	**Yes**
`git add`, `git commit` on `build-first-steps`	plain **Yes**
An install it explained	plain **Yes** (WSL `sudo`: No, run it yourself)
An install it didn't explain	**No**: "Explain this command first"
Anything that **changes** `main` (switch, checkout, merge, push, `branch -f`)	**No**
Read-only: `git diff`, `git log`, `git merge-base`, `git status`	**Yes**
`git push`	**No** (you push in step 7)
`--force`, `reset --hard`, `git clean`, `rm -rf`	**No**
At the end: any command with `rm -rf .superpowers/sdd/...`, even inside a longer one	**No** is fine: "Carry on without deleting it."

It only stops by itself for: can't-be-undone, security-sensitive, outside your project (push, merge, publish),

The ledger: stop and carry on

`.superpowers/sdd/<plan-name>/progress.md`, in your project folder

- `# SDD ledger`, then `Task 1: complete` ... as tasks finish
- git ignores it; Superpowers may ask to delete the folder when the build finishes, often inside a longer command: read it all, and say No to `rm -rf`

Stop safely: just after a task finishes, `Esc`, then `/exit`.

Carry on (being tested): `claude` in `~/semester-project`, on `build-first-steps`:

```
/superpowers:subagent-driven-development Continue the plan from the
ledger. The plan is docs/superpowers/plans/<your-plan-file>.md.
```

A worktree made anyway? git worktree list. On exit, choose Keep, never remove.

Usage limits (being tested)

- Pro: a **five-hour session** limit and a **weekly** limit, shared by Claude and Claude Code. No published numbers.
- **Subagents use more:** their requests "count toward the same usage limits as your main conversation".
- `/usage`: usage bars, reset times, the subagents' share (`d` / `w`). The help article names `/status`.
- At the limit: `You've hit your session limit · resets 3:45pm`
- **Wait for the reset.** Leave Claude Code open and it continues by itself, or come back and "Continue the plan from the ledger".
- **Start early in the week.**

support.claude.com/en/articles/11145838 and code.claude.com/docs/en/costs

Finish

```
Implementation complete. What would you like to do?
1. Merge back to main locally
2. Push and create a Pull Request
3. Keep the branch as-is (I'll handle it later)
```

Don't answer yet. In a second terminal window:

1. Run the tests yourself. **Screenshot.**
2. Run your program on **your own example**; compare with your hand-worked answer. **Screenshot.**

Then answer `2`. Never 1. Plain **Yes** to `git push` and `gh pr create`. Copy the PR address.

Merge, pull, check

On GitHub: **Files changed** shows only the plan file, `.gitignore`, code and tests (no `.venv/`, `node_modules/`, `__pycache__/`), then **Merge pull request**, **Confirm merge**.

In Claude Code:

```
Switch back to main and pull the latest changes from GitHub.
```

Done when:

- the PR is merged, with the plan file in `docs/superpowers/plans/`
- on `main`, your tests pass
- your program matches your hand-worked example

Keep your local build-first-steps branch until graded. Say No to deleting a branch or worktree. Committed to main? README rescue steps, typed by you.

Assignment

- The **merged PR** address: plan file and the three build steps
- Screenshot: **tests passing**
- Screenshot: **your program on your own example**, hand-worked answer beside it
- Reflection: **one review finding** and what changed; roughly **how much usage** the build took

Criterion	Points
Merged PR: plan file (1), three steps work (2), branch and PR (1)	4
Tests pass, and checked against a hand-worked example	3
Reflection: review finding (2), usage (1)	3

Next week: Reviewing code

You learn to read and review what Claude built.

Bring:

- your merged first three steps
- the review finding you noted
- your `/usage` numbers from this week